

CODEALPHA DATA ANALYTICS INTERNSHIP

Task 1: web scraping-Detailed report

Name: BAIRU SAI KIRAN

Internship ID :CA/DF1/53948

GitHub Repo: code Alpha_ Web Scraping

1.objective

To use python libraries like Beautiful soup and scrapy to extract structured data from public website , handle HTML parsing and web navigation ,and create a custom dataset Tailored for analysis

2 Tools & technologies used

Components	Technology
languages	Python 3.10
Web scraping	Beautifulsoup4,requests, pandas
Alternative tool	Scrapy for large- scale crawling
No-code option	Octoparse/parseHu if coding not preferred
Data storage	CSV, excel, SQLite

3. project idea: scrape E-commerce product data

For this task, I scraped product data from a public demo e-commerce site

Books.toscrape.com.this site is legal for scraping practice.

Data extracted:

1.book title

2.price

3. stock availability

4. star rating

5.product URL

This create a custom dataset for price analysis,which can be used in task 2 EDA

4. methodology & code implementation

4.1 understanding HTML structure

Used browser DevTools to in inspect the site.

Each book is inside<article

Class= "product_pod">.price is in <p

Class= "price_colour">.rating in <p

Class= "star-rating Three">.

4.2 python code with beautifullsoup

File:scraper.py

< > python code

import requests

from bs4 import BeautifulSoup

import pandas as pd

import time

```
base_url = "http://books.toscrape.com/catalogue/page-{}.html"
all_books = []
```

```
# Scrape first 5 pages as demo
```

```
for page in range(1, 6):
```

```
    url = base_url.format(page)
```

```
    response = requests.get(url)
```

```
    soup = BeautifulSoup(response.text, 'html.parser')
```

```
    books = soup.find_all('article', class_='product_pod')
```

```
    for book in books:
```

```
        title = book.h3.a['title']
```

```
        price = book.find('p', class_='price_color').text
```

```
        stock = book.find('p', class_='instock
availability').text.strip()
```

```
        rating = book.find('p', class_='star-rating')['class'][1] #
Three, Four, etc.
```

```
        link = "http://books.toscrape.com/catalogue/" +
book.h3.a['href']
```

```
        all_books.append({
```

```
            "Title": title,
```

```
"Price": price,  
"Stock": stock,  
"Rating": rating,  
"URL": link  
})
```

```
time.sleep(1) # Respect server - don't overload
```

```
# Save to CSV
```

```
df = pd.DataFrame(all_books)
```

```
df.to_csv("books_dataset.csv", index=False)
```

```
print(f"Scraped {len(df)} books. Saved to books_dataset.csv")
```

```
file: requirement.txt
```

```
requests
```

```
beatifulsoup4
```

```
pandas
```

```
lxml
```

4.3 handling pagination & navigation

The script loops through page-{}. Html from 1to 5. For dynamic sites, scrapy's

Crawlspider or selenium world handle "next" button clicks.

4.4 data cleaning

< > python code

Clean price column: "£51.77" -> 51.77

```
df['Price'] = df['Price'].str.replace('£', "").astype(float)
```

```
df['Rating'] = df['Rating'].map({'One':1, 'Two':2, 'Three':3, 'Four':4, 'Five':5})
```

5. result

Metric	value
Total records scraped	100 books from 5 pages
Columns created	5: title, price, stock, rating ,URL
Output format	Books_dataset.csv
Time taken	8 seconds with 1s delay per page

Sample output:

Tittle	price	stock	rating	URL
A light in the attic	51.77	In stock	3	Books.toscrap e..
Tipping the velvet	53.74	In stock	1	Books.toscrap e..

6. challenge & solutions

challenge	solutions
Website blocks requests	Added headers={'user-agent':'mozilla/5.0'}
Inconsistent HTML tag	Used,find() with check for none before.text
Dynamic javascript content	For JS-heavy sites, use selenium or scrapy-splash

7. Ethical Web Scraping Practices

1. Checked `http://robots.txt: books.toscrape.com/robots.txt` allows scraping
2. Rate limiting: 1 second delay between requests
3. Public data only: No login or personal data collected
4. Terms of Service: Used only for educational purpose

8. Submission Checklist for CodeAlpha

1. GitHub Repo: CodeAlpha_WebScraping with `scraper.py`, `books_dataset.csv`, `README.md`
2. LinkedIn Video: 60-sec screen record showing code running + CSV output. Tag @CodeAlpha
3. PDF Report: This document with screenshots of terminal output + first 10 rows of CSV
4. Form Submission: Upload PDF + GitHub link via WhatsApp group form

9. How to Get Appreciation

1. Add visualizations: Use the scraped CSV for a quick Matplotlib bar chart of "Avg Price by Rating" and put it in PDF
2. Deploy dataset on Kaggle: Upload `books_dataset.csv` to Kaggle and link it. Shows initiative
3. Scrape 2 sites: Add one more simple site to show you can handle different HTML structures

4. Write "Learning Outcome": "Learned HTML parsing, handling pagination, rate limiting, and creating analysis-ready datasets from unstructured web data"